Day 61 — AI Monitoring & Observability

Objective

Provide traceable operational visibility across ATS, screening, interview, scoring and recommendation components.

Event model

Use `utils.ai_observability.StructuredEventLogger` for JSON event records with:

• `event_id`, UTC `timestamp`, `event_type`, `service`, `status`

• `correlation_id` to connect one request across stages

• pseudonymous `candidate_ref` (never a candidate name or email)

• `duration_ms` where measured

• `details` after sensitive-key redaction

Recommended event types: `api_request`, `resume_job_started`, `resume_job_completed`, `score_generated`, `interview_stage_completed`, `recommendation_created`, `validation_error`, `processing_error`, `audit_event`.

Do not log resume text, raw transcripts, interview answers, contact details, credentials, access tokens, or uploaded file bytes. The helper redacts known sensitive keys, but production deployments still need schema allow-lists and security review.

Metrics

Metric	Definition	Collection
Request count	Requests by service/route/status	API middleware or gateway
Response time	End-to-end elapsed milliseconds	monotonic timer around request
p95 latency	95th percentile of request duration	metrics backend
Failure rate	failed requests / all requests × 100	status/error counters
Stage completion	completed / started by pipeline stage	correlation events
Candidate processing	submitted, parsed, scored, failed counts	job events
Interview completion	completed sessions / started sessions	interview lifecycle events
Score distribution	aggregate score bands by model version	privacy-reviewed aggregate only
Human override rate	human changes to AI recommendation / reviewed cases	audit events

Accuracy is not an operational counter. Measure it only against representative, consented and independently labelled validation data using agreed metrics (precision, recall, F1, calibration and subgroup error analysis). Never infer accuracy from agreement with one demo reviewer.

Alert rules (initial proposal)

The `AlertRuleEvaluator` supports configurable numeric thresholds. Initial values below are starting points to tune after baseline measurement, not observed service SLAs:

• failure rate > 5% over a rolling 5-minute window: critical

• p95 latency > 2,000 ms for 10 minutes: warning

• API errors > 20 over 5 minutes: warning

• any audit-write failure: critical (wire through durable audit storage)

• queue age > agreed operational target: warning/critical by severity

The in-process evaluator does not implement rolling windows, paging, or persistence. A production metrics/alerting platform must provide those.

Dashboard structure

1. **System health:** request volume, error rate, p50/p95 latency, uptime, active jobs.

2. **Pipeline funnel:** uploaded → parsed → ATS scored → screening completed → HR completed → technical completed → recommendation generated.

3. **Interview operations:** started/completed sessions, abandonment, stage duration, failures.

4. **Scoring quality:** score distributions, missing-round coverage, reviewer overrides, validated model metrics.

5. **Governance:** consent coverage, audit-write status, access denials, retention/deletion backlog.

Avoid displaying individual candidate details on broad operational dashboards. Apply role-based access, minimum cohort sizes and export controls.

Decision audit

Record actor reference, action, candidate reference, timestamp, model/rule version, score, recommendation, rationale codes and whether a human reviewer confirmed/overrode it. Keep raw evidence in separately access-controlled storage and link by protected reference only. Use `DecisionAuditTrail` as a local test/reference helper; it is not durable or tamper-evident.

Implementation boundary

This batch adds dependency-free in-process helpers. Existing `utils.logger` remains the application's current file logger. No API middleware, external metrics exporter, dashboard UI, alert delivery, durable audit database, or production retention enforcement is deployed by this change.